P0192R4
`short float` and fixed-size floating point types

Published Proposal, 2018-10-08

This version:
https://wg21.link/P0192R4

Issue Tracking:
Inline In Spec

Authors:
Michał Dominiak (NVIDIA)
Bryce Adelstein Lelbach (NVIDIA)
Boris Fomitchev (NVIDIA)
Sergei Nikolaev (NVIDIA)

Audience:
EWG, LEWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

§ 1. Abstract

This proposal follows P0192R1 in proposing a new fundamental type, `short float`: a floating point type of unspecified length, shorter or equal to that of `float`. In addition, it also proposes standard library

aliases for fixed width floating point types, required to conform to [IEEE-754-2008], including `std::float16_t`. Library support for `short float` and `std::float16_t` is also included.

§ 2. Motivation

One may wonder: why would a programming language need yet another floating point type after so many years of doing just fine without it? Apparently, the times are (a-)changing. Small binary floating-point representation demand and support are becoming more and more common rapidly. Efficient support for hardware that use the new formats becomes mission critical for major software products.

§ 2.1. Application use is growing

Application areas include computer graphics, image representation and machine learning. For example, a 16-bit floating-point number better represents the dynamic range of images than 16-bit or even 32-bit integers. A 16-bit floating-point number adequately handles human perceptual range. In 2012, Adobe has defined new HDR [DNG] image file format that most commonly uses 16-bit floats. This gives those 16-bits much more dynamic range than a traditional file stored as 16 or 32-bit integer data. Both Photoshop and Lightroom, as well as every professional camera produced in 2015 make use of it.

§ 2.2. Software support is growing

- OpenGL has `GL_HALF_FLOAT` format since [OpenGL3.0].
- The [OpenEXR] software distribution includes `Half`, a C++ class for manipulating half float values almost as if they were a built-in C++ data type.
- NVIDIA's [CUDA7.5] platform header `cuda_fp16.h` defines the `half` and `half2` data types and defines `__half2float()` and `__float2half()` for conversion between that and `float`. [4]
- The GCC compiler provides an `__fp16` native data type extension for ARM. Values of that type are promoted to `float` for computation ([GCCFP16]).
- The LLVM IR provides a 16-bit floating-point type called `half`.
- Recently, a new 16-bit float format appeared, called [bfloat16] - truncated 32-bit float, used in Google TPUs and TensorFlow `half`.

§ 2.3. Hardware support is growing

- NVIDIA was the first to implement 16-bit floating point in silicon, with the GeForce FX, released in late 2002.
- Intel provides instructions for converting between 16-bit and 32-bit floats and C-level intrinsics to use them (see [INTEL-HALF-PERF], an Intel article on half precision performance benefits).
- ARM provides support as an optional extension to the VFPv3 architecture: [ARM-HF].

Defining standards for 16-bit float math already exist. The [IEEE-754-2008] standard defined `binary16` 16-bit float in 2008. ISO/IEC 60559 ratified that standard in 2011. However, support for just the IEEE `binary16` format does not cover existing use cases.

- OpenGL provides 11-bit and 10-bit float channels in `GL_R11F_G11F_B10F` and 14-bit float in `GL_RGB9_E5`.
- ARM, along an [IEEE-754-2008] compatible 16-bit floating-point format, provides a 16-bit floating-point format that differs from IEEE 'binary16' by dropping support for NaN and Infinity and then extending the range of values.
- The TI MSP430X architecture provides a 20-bit word-addressed machine. Short floating-point support on that machine would naturally use a 20-bit format.
- Google TPU and TensorFlow use [bfloat16].

3. Proposed solution

We propose adding a new fundamental type, `short float` - for a floating-point type of unspecified (platform defined) bit size, shorter or equal to that of `float`. Language needs `short float` to represent "shorter than `float`" math that may be natively available on the platform. This name looks intuitive via `short int` analogy. Most important, it does not introduce any new keywords.

The proposed suffixes for `short float` literals are `sf` and `SF`.

This proposal also extends the definition of *floating-point promotions*, including a promotion from `short float` to `float`.

We also propose adding a set of conditionally supported type aliases in namespace `std`: `float16_t`, `float32_t` and `float64_t`. Those types would be quaranteed to be respectively 16, 32 and 64 bit long in their representation, and would be required to implement [IEEE-754-2008] `binary16`, `binary32` and `binary64` formats, respectively. We propose to put those aliases into a new header, `<cstdfloat>` for consistency with `<cstdint>`, and to make this new header a freestanding header.

Several people have suggested not having a new fundamental type, but only exposing it as the `std::float16_t` alias. This approach problems. All the library changes would still need to be done, but substituting `short float` with `std::float16_t`, plus additional wording gauranteeing that the additional overloads don't exist if, for some reason, `float` has 16 bits. Additionally, the definition of a floating-point type would still need to be extended to include that - conditionally supported! - type, and to give it the same capabilities that other floating-point types have. The authors of this paper find this to be a change that is almost as complex as adding a new fundamental type, but with more corner cases around overload resolution for standard library functions.

3.1. Implementation options

As of storage and bit-layout for a short float number, we would expect most implementations to follow [IEEE-754-2008] or [bfloat16] half-precision floating point number formats. On platform that do not provide any advantages of using shorter float, short float may be implemented as storage-only type, like `__fp16` on GCC/ARM today. For example, it can be stored in `bfloat16` format in memory (occupying less bytes than `float`), converted to native 32-bit `float` on read from memory, operated on using native 32-bit floating-point math operations and converted back to `bfloat16` on store to memory. Or, the platform may choose to not take any advantage of `short float` and represent it using `float` in both memory and registers.

3.2. Implementation experience

Since CUDA 7.5 introduction of `half` 16-bit floating type, applications can benefit by storing up to 2x larger models in GPU memory. Applications that are bottlenecked by memory bandwidth may get up to 2x speedup. And applications bottlenecked by FP32 computation may benefit from 2x faster computation on `half2` data. NVIDIA GPUs implement the [IEEE-754-2008] floating point standard, which defines half-precision numbers as follows:

- Sign: 1 bit
- Exponent width: 5 bits
- Significand precision: 11 bits (10 explicitly stored)

Google TPUs implement `bfloat16` format, which defines half-precision numbers as higher 16 bits of 32-bit IEEE float:

- Sign: 1 bit
- Exponent width: 8 bits
- Significand precision: 8 bits (7 explicitly stored)

4. Proposed wording

4.1. Wording for a new fundamental type

4.1.1. Core language

Modify Floating literals **[lex.fcon]** by adding short float suffixes to *floating-suffix*:

> *floating-suffix:* one of
>
> sf f l SF F L

In paragraph 1, modify sentence 13:

> The type of a floating literal is `double` unless explicitly specified by a suffix. The suffixes `sf` and `SF` specify `short float`, the suffixes `f` and `F` specify `float`, the suffixes `l` and `L` specify `long double`. [...]

Modify Fundamental types **[basic.fundamental]** paragraph 8:

> There are ~~three~~ four *floating-point types*: `short float`, `float`, `double`, and `long double`. The type `float` provides at least as much precision as `short float`, the type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `short float` is a subset of the set of values of the type `double`, the set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. The value representation of floating-point types is implementation-defined. [...]

Modify Floating-point promotion **[conv.fpprom]** as follows:

> A prvalue of type `short float` can be converted to a prvalue of type `float`. The value is unchanged.
>
> A prvalue of type `float` can be converted to a prvalue of type `double`. The value is unchanged.
>
> ~~This conversion is~~ These conversions are called *floating-point ~~promotion~~promotions*.

Modify Usual arithmetic conversions **[expr.arith.conv]** paragraph 1 as follows:

> [...]
>
> - If either operand is of type `long double`, the other shall be converted to `long double`.
> - Otherwise, if either operand is `float`, the other shall be converted to `float`.
> - Otherwise, if either operand is `short float`, the other shall be converted to `short float`.
> - Otherwise, the integral promotions shall be performed on both operands. [...]

In Simple type specifiers **[dcl.type.simple]**, modify table 11 as follows:

> | Specifier(s) | Type |
> |---|---|
> | [...] | [...] |
> | `wchar_t` | "`wchar_t`" |
> | `short float` | "`short float`" |
> | `float` | "`float`" |
> | [...] | [...] |

Modify List-initialization **[dcl.init.list]** paragraph 7 item 2:

> - from ~~long double to double or float, or from double to float,~~ higher precision floating-point type to a lower precision one, except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly), or

In Standard conversion sequences [**over.ics.scs**], modify table 13 as follows:

> [...] Floating-point ~~promotion~~ promotions [...]

In Predefined macro names [**cpp.predefined**], modify table 16 as follows:

> | Macro name | Value |
> |---|---|
> | [...] | [...] |
> | `__cpp_rvalue_references` | `200610L` |
> | `__cpp_short_float` | `201810L` |
> | `__cpp_sized_deallocation` | `201309L` |
> | [...] | [...] |

4.1.2. Library wording

Modify Header `<cstdlib>` synopsis [**cstdlib.syn**] as follows:

> [...]
> ```
> long long int abs(long long int j);
> short float abs(short float j);
> float abs(float j);
> ```
> [...]

> Note: `strtosf` is not added on purpose, since the `strto*` family is owned by C. `abs` is provided for the new type, since C++ already extends the overload set of this function.

Modify Header `<limits>` synopsis [**limits.syn**] as follows:

> [...]
> ```
> template<> class numeric_limits<unsigned long long>;
>
> template<> class numeric_limits<short float>;
> template<> class numeric_limits<float>;
> ```
> [...]

Do not modify Header `<cfloat>` synopsis [**cfloat.syn**].

> Note: no macros are added to `<cfloat>`, on purpose, because that header's contents are fully owned by C.

Modify Header `<charconv>` synopsis [**charconv.syn**] as follows:

> ISSUE 1 it is possible that instead of adding a new overload, the overloads could be folded into a single specified function, the way that integer overloads of all of these are specified.

[...]

```
// [charconv.to.chars], primitive numerical output conversion
struct to_chars_result {
  char* ptr;
  errc ec;
};

to_chars_result to_chars(char* first, char* last, *see below* value, int base = 10);
```

```
to_chars_result to_chars(char* first, char* last, short float value);
```

```
to_chars_result to_chars(char* first, char* last, float value);
to_chars_result to_chars(char* first, char* last, double value);
to_chars_result to_chars(char* first, char* last, long double value);
```

```
to_chars_result to_chars(char* first, char* last, short float value, chars_format fmt);
```

```
to_chars_result to_chars(char* first, char* last, float value, chars_format fmt);
to_chars_result to_chars(char* first, char* last, double value, chars_format fmt);
to_chars_result to_chars(char* first, char* last, long double value, chars_format fmt);
```

```
to_chars_result to_chars(char* first, char* last, short float value,
                         chars_format fmt, int precision);
```

```
to_chars_result to_chars(char* first, char* last, float value,
                         chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, double value,
                         chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, long double value,
                         chars_format fmt, int precision);

// [charconv.from.chars], primitive numerical input conversion
struct from_chars_result {
  const char* ptr;
  errc ec;
};

from_chars_result from_chars(const char* first, const char* last,
                             see below& value, int base = 10);
```

```
from_chars_result from_chars(const char* first, const char* last, short float& value,
                             chars_format fmt = chars_format::general);
```

```
from_chars_result from_chars(const char* first, const char* last, float& value,
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, double& value,
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, long double& value,
                             chars_format fmt = chars_format::general);
}
```

Modify Primitive numeric output conversion **[charconv.to.chars]** as follows, by adding overloads to the lists of signatures:

[...]

```
to_chars_result to_chars(char* first, char* last, short float value);
```

```
to_chars_result to_chars(char* first, char* last, float value);
to_chars_result to_chars(char* first, char* last, double value);
to_chars_result to_chars(char* first, char* last, long double value);
```

[...]

```
to_chars_result to_chars(char* first, char* last, short float value, chars_format fmt);
```

```
to_chars_result to_chars(char* first, char* last, float value, chars_format fmt);
to_chars_result to_chars(char* first, char* last, double value, chars_format fmt);
to_chars_result to_chars(char* first, char* last, long double value, chars_format fmt);
```

[...]

```
to_chars_result to_chars(char* first, char* last, short float value,
                         chars_format fmt, int precision);
```

```
to_chars_result to_chars(char* first, char* last, float value,
                         chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, double value,
                         chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, long double value,
                         chars_format fmt, int precision);
```

[...]

Note: no changes in descriptions are needed, since all those overloads of every of those functions are specified together already.

Modify Primitive numeric input conversions **[charconv.from.chars]** as follows, by adding an overload to the list of signatures:

[...]

```
from_chars_result from_chars(const char* first, const char* last, short float& value,
                             chars_format fmt = chars_format::general);
```

```
from_chars_result from_chars(const char* first, const char* last, float& value,
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, double& value,
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, long double& value,
                             chars_format fmt = chars_format::general);
```

[...]

Note: no changes in descriptions are needed, since all those overloads of every of those functions are specified together already.

Modify Header `<string>` synopsis **[string.syn]** as follows:

[...]

```
string to_string(unsigned long long val);
string to_string(short float val);
string to_string(float val);
```

[...]

```
wstring to_wstring(unsigned long long val);
wstring to_wstring(short float val);
wstring to_wstring(float val);
```

[...]

ISSUE 2 the definitions of the `sto*` family depend on C functions in the `strto*` family. Should an overload for `short float` be added?

Modify Numeric conversions [**string.conversions**] as follows:

[...]

```
string to_string(int val);
string to_string(unsigned val);
string to_string(long val);
string to_string(unsigned long val);
string to_string(long long val);
string to_string(unsigned long long val);
string to_string(short float val);
string to_string(float val);
string to_string(double val);
string to_string(long double val);
```

7. Returns: Each function returns a `string` object holding the character representation of the value of its argument that would be generated by calling `sprintf(buf, fmt, val)` with a format specifier of `"%d"`, `"%u"`, `"%ld"`, `"%lu"`, `"%lld"`, `"%llu"`, `"%f"`, `"%f"`, `"%f"`, or `"%Lf"`, respectively, where `buf` designates an internal character buffer of sufficient size.

[...]

```
wstring to_wstring(int val);
wstring to_wstring(unsigned val);
wstring to_wstring(long val);
wstring to_wstring(unsigned long val);
wstring to_wstring(long long val);
wstring to_wstring(unsigned long long val);
wstring to_wstring(short float val);
wstring to_wstring(float val);
wstring to_wstring(double val);
wstring to_wstring(long double val);
```

14. Returns: Each function returns a `wstring` object holding the character representation of the value of its argument that would be generated by calling `sprintf(buf, fmt, val)` with a format specifier of `"%d"`, `"%u"`, `"%ld"`, `"%lu"`, `"%lld"`, `"%llu"`, `"%f"`, `"%f"`, `"%f"`, or `"%Lf"`, respectively, where `buf` designates an internal character buffer of sufficient size.

Modify Complex numbers [**complex.numbers**] paragraph 2:

The effect of instantiating the template `complex` for any type other than `short float`, `float`, `double`, or `long double` is unspecified. The specializations `complex<short float>`, `complex<float>`, `complex<double>`, and `complex<long double>` are literal types.

Modify Header `<complex>` synopsis **[complex.syn]** as follows:

> [...]
>
> ```
> // [complex.special], complex specializations
> ```
>
> ```
> template<> class complex<short float>;
> ```
>
> ```
> template<> class complex<float>;
> template<> class complex<double>;
> template<> class complex<long double>;
> ```
>
> [...]
>
> ```
> // [complex.literals], complex literals
> inline namespace literals {
> inline namespace complex_literals {
>   constexpr complex<long double> operator""il(long double);
>   constexpr complex<long double> operator""il(unsigned long long);
>   constexpr complex<double> operator""i(long double);
>   constexpr complex<double> operator""i(unsigned long long);
>   constexpr complex<float> operator""if(long double);
>   constexpr complex<float> operator""if(unsigned long long);
> ```
>
> ```
>   constexpr complex<short float> operator""isf(long double);
>   constexpr complex<short float> operator""isf(unsigned long long);
> ```
>
> ```
> }
> }
> ```

Modify `complex` specializations **[complex.special]** as follows:

```
namespace std {
  template<> class complex<short float> {
  public:
    using value_type = short float;

    constexpr complex(short float re = 0.0sf, short float im = 0.0sf);
    constexpr complex(const complex<short float>&) = default;
    constexpr explicit complex(const complex<float>&);
    constexpr explicit complex(const complex<double>&);
    constexpr explicit complex(const complex<long double>&);

    constexpr short float real() const;
    constexpr void real(short float);
    constexpr short float imag() const;
    constexpr void imag(short float);

    constexpr complex& operator= (short float);
    constexpr complex& operator+=(short float);
    constexpr complex& operator-=(short float);
    constexpr complex& operator*=(short float);
    constexpr complex& operator/=(short float);

    constexpr complex& operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };
```

```
  template<> class complex<float> {
  public:
    using value_type = float;

    constexpr complex(float re = 0.0f, float im = 0.0f);
```

```
    constexpr complex(const complex<short float>&);
```

```
  constexpr complex(const complex<float>&) = default;
  constexpr explicit complex(const complex<double>&);
  constexpr explicit complex(const complex<long double>&);

  constexpr float real() const;
  constexpr void real(float);
  constexpr float imag() const;
  constexpr void imag(float);

  constexpr complex& operator= (float);
  constexpr complex& operator+=(float);
  constexpr complex& operator-=(float);
  constexpr complex& operator*=(float);
  constexpr complex& operator/=(float);

  constexpr complex& operator=(const complex&);
  template<class X> constexpr complex& operator= (const complex<X>&);
  template<class X> constexpr complex& operator+=(const complex<X>&);
  template<class X> constexpr complex& operator-=(const complex<X>&);
  template<class X> constexpr complex& operator*=(const complex<X>&);
  template<class X> constexpr complex& operator/=(const complex<X>&);
};

template<> class complex<double> {
public:
  using value_type = double;

  constexpr complex(double re = 0.0, double im = 0.0);
```

```
  constexpr complex(const complex<short float>&);
```

```
  constexpr complex(const complex<float>&);
  constexpr complex(const complex<double>&) = default;
  constexpr explicit complex(const complex<long double>&);

  constexpr double real() const;
  constexpr void real(double);
  constexpr double imag() const;
  constexpr void imag(double);

  constexpr complex& operator= (double);
  constexpr complex& operator+=(double);
  constexpr complex& operator-=(double);
  constexpr complex& operator*=(double);
  constexpr complex& operator/=(double);

  constexpr complex& operator=(const complex&);
  template<class X> constexpr complex& operator= (const complex<X>&);
  template<class X> constexpr complex& operator+=(const complex<X>&);
  template<class X> constexpr complex& operator-=(const complex<X>&);
  template<class X> constexpr complex& operator*=(const complex<X>&);
  template<class X> constexpr complex& operator/=(const complex<X>&);
};

template<> class complex<long double> {
public:
  using value_type = long double;

  constexpr complex(long double re = 0.0L, long double im = 0.0L);
```

```
  constexpr complex(const complex<short float>&);
```

```
    constexpr complex(const complex<float>&);
    constexpr complex(const complex<double>&);
    constexpr complex(const complex<long double>&) = default;

    constexpr long double real() const;
    constexpr void real(long double);
    constexpr long double imag() const;
    constexpr void imag(long double);

    constexpr complex& operator= (long double);
    constexpr complex& operator+=(long double);
    constexpr complex& operator-=(long double);
    constexpr complex& operator*=(long double);
    constexpr complex& operator/=(long double);

    constexpr complex& operator=(const complex&);
    template<class X> constexpr complex& operator= (const complex<X>&);
    template<class X> constexpr complex& operator+=(const complex<X>&);
    template<class X> constexpr complex& operator-=(const complex<X>&);
    template<class X> constexpr complex& operator*=(const complex<X>&);
    template<class X> constexpr complex& operator/=(const complex<X>&);
  };
}
```

Add an item to Additional overloads **[cmplx.over]** paragraph 2:

> [...]
>
> - Otherwise, if the argument has type `float`, then it is effectively cast to `complex<float>`.
> - Otherwise, if the argument has type `short float`, then it is effectively cast to `complex<short float>`.

Add an item to Additional overloads **[cmplx.over]** paragraph 3:

> [...]
>
> - Otherwise, if either argument has type `complex<float>` or `float`, then both arguments are effectively cast to `complex<float>`.
> - Otherwise, if either argument has type `complex<short float>` org `short float`, then both arguments are effectively cast to `complex<short float>`.

Modify Suffixes for complex number literals **[complex.literals]** as follows:

1. This subclause describes literal suffixes for constructing complex number literals. The suffixes ~~`i`, `il`, and `if`~~ `il`, `i`, `if`, and `isf` create complex numbers of the types ~~`complex<double>`, `complex<long double>`, and~~ `complex<long double>`, `complex<double>`, `complex<float>`, and `complex<short float>` respectively, with their imaginary part denoted by the given literal number and the real part being zero.

```
constexpr complex<long double> operator""il(long double d);
constexpr complex<long double> operator""il(unsigned long long d);
```

2. Returns: `complex<long double>{0.0L, static_cast<long double>(d)}`.

```
constexpr complex<double> operator""i(long double d);
constexpr complex<double> operator""i(unsigned long long d);
```

3. Returns: `complex<double>{0.0, static_cast<double>(d)}`.

```
constexpr complex<float> operator""if(long double d);
constexpr complex<float> operator""if(unsigned long long d);
```

4. Returns: `complex<float>{0.0f, static_cast<float>(d)}`.

```
constexpr complex<short float> operator""isf(long double d);
constexpr complex<short float> operator""isf(unsigned long long d);
```

5. Returns: `complex<short float>{0.0sf, static_cast<short float>(d)}`.

Modify General requirements [**rand.req.genl**] paragraph 1 d):

d) that has a template type parameter named `RealType` is undefined unless the corresponding template argument is cv-unqualified and is one of `short float`, `float`, `double`, or `long double`.

Modify Header `<cmath>` synopsis [**cmath.syn**] as follows:

[...]

```
namespace std {
```

```
  short float acos(short float x); // see [library.c]
```

```
  float acos(float x);  // see [library.c]
  double acos(double x);
  long double acos(long double x);  // see [library.c]
  float acosf(float x);
  long double acosl(long double x);
```

```
  short float asin(short float x);  // see [library.c]
```

```
  float asin(float x);  // see [library.c]
  double asin(double x);
  long double asin(long double x);  // see [library.c]
  float asinf(float x);
  long double asinl(long double x);
```

```
  short float atan(short float x);  // see [library.c]
```

```
float atan(float x);  // see [library.c]
double atan(double x);
long double atan(long double x);  // see [library.c]
float atanf(float x);
long double atanl(long double x);
```

```
short float atan2(short float y, short float x);  // see [library.c]
```

```
float atan2(float y, float x);  // see [library.c]
double atan2(double y, double x);
long double atan2(long double y, long double x);  // see [library.c]
float atan2f(float y, float x);
long double atan2l(long double y, long double x);
```

```
short float cos(short float x);  // see [library.c]
```

```
float cos(float x);  // see [library.c]
double cos(double x);
long double cos(long double x);  // see [library.c]
float cosf(float x);
long double cosl(long double x);
```

```
short float sin(short float x);  // see [library.c]
```

```
float sin(float x);  // see [library.c]
double sin(double x);
long double sin(long double x);  // see [library.c]
float sinf(float x);
long double sinl(long double x);
```

```
short float tan(short float x);  // see [library.c]
```

```
float tan(float x);  // see [library.c]
double tan(double x);
long double tan(long double x);  // see [library.c]
float tanf(float x);
long double tanl(long double x);
```

```
short float acosh(short float x);  // see [library.c]
```

```
float acosh(float x);  // see [library.c]
double acosh(double x);
long double acosh(long double x);  // see [library.c]
float acoshf(float x);
long double acoshl(long double x);
```

```
short float asinh(short float x);  // see [library.c]
```

```
float asinh(float x);  // see [library.c]
double asinh(double x);
long double asinh(long double x);  // see [library.c]
float asinhf(float x);
long double asinhl(long double x);
```

```
short float atanh(short float x);  // see [library.c]
```

```
float atanh(float x);  // see [library.c]
double atanh(double x);
long double atanh(long double x);  // see [library.c]
float atanhf(float x);
long double atanhl(long double x);
```

```
short float cosh(short float x);  // see [library.c]
```

```
float cosh(float x);  // see [library.c]
double cosh(double x);
long double cosh(long double x);  // see [library.c]
float coshf(float x);
long double coshl(long double x);
```

```
short float sinh(short float x);  // see [library.c]
```

```
float sinh(float x);  // see [library.c]
double sinh(double x);
long double sinh(long double x);  // see [library.c]
float sinhf(float x);
long double sinhl(long double x);
```

```
short float tanh(short float x);  // see [library.c]
```

```
float tanh(float x);  // see [library.c]
double tanh(double x);
long double tanh(long double x);  // see [library.c]
float tanhf(float x);
long double tanhl(long double x);
```

```
short float exp(short float x);  // see [library.c]
```

```
float exp(float x);  // see [library.c]
double exp(double x);
long double exp(long double x);  // see [library.c]
float expf(float x);
long double expl(long double x);
```

```
short float exp2(short float x);  // see [library.c]
```

```
float exp2(float x);  // see [library.c]
double exp2(double x);
long double exp2(long double x);  // see [library.c]
float exp2f(float x);
long double exp2l(long double x);
```

```
short float expm1(short float x);  // see [library.c]
```

```
float expm1(float x);  // see [library.c]
double expm1(double x);
long double expm1(long double x);  // see [library.c]
float expm1f(float x);
long double expm1l(long double x);
```

```
short float frexp(short float value, int* exp);  // see [library.c]
```

```
float frexp(float value, int* exp);  // see [library.c]
double frexp(double value, int* exp);
long double frexp(long double value, int* exp);  // see [library.c]
float frexpf(float value, int* exp);
long double frexpl(long double value, int* exp);
```

```
int ilogb(short float x);  // see [library.c]
```

```
int ilogb(float x);  // see [library.c]
int ilogb(double x);
int ilogb(long double x);  // see [library.c]
int ilogbf(float x);
int ilogbl(long double x);
```

```
short float ldexp(short float x, int exp);  // see [library.c]
```

```
float ldexp(float x, int exp);  // see [library.c]
double ldexp(double x, int exp);
long double ldexp(long double x, int exp);  // see [library.c]
float ldexpf(float x, int exp);
long double ldexpl(long double x, int exp);
```

```
short float log(short float x);  // see [library.c]
```

```
float log(float x);  // see [library.c]
double log(double x);
long double log(long double x);  // see [library.c]
float logf(float x);
long double logl(long double x);
```

```
short float log10(short float x);  // see [library.c]
```

```
float log10(float x);  // see [library.c]
double log10(double x);
long double log10(long double x);  // see [library.c]
float log10f(float x);
long double log10l(long double x);
```

```
short float log1p(short float x);  // see [library.c]
```

```
float log1p(float x);  // see [library.c]
double log1p(double x);
long double log1p(long double x);  // see [library.c]
float log1pf(float x);
long double log1pl(long double x);
```

```
short float log2(short float x);  // see [library.c]
```

```
float log2(float x);  // see [library.c]
double log2(double x);
long double log2(long double x);  // see [library.c]
float log2f(float x);
long double log2l(long double x);
```

```
short float logb(short float x);  // see [library.c]
```

```
float logb(float x);  // see [library.c]
double logb(double x);
long double logb(long double x);  // see [library.c]
float logbf(float x);
long double logbl(long double x);
```

```
short float modf(short float value, short float* iptr);  // see [library.c]
```

```
float modf(float value, float* iptr);  // see [library.c]
double modf(double value, double* iptr);
long double modf(long double value, long double* iptr);  // see [library.c]
float modff(float value, float* iptr);
long double modfl(long double value, long double* iptr);
```

```
short float scalbn(short float x, int n);  // see [library.c]
```

```
float scalbn(float x, int n);  // see [library.c]
double scalbn(double x, int n);
long double scalbn(long double x, int n);  // see [library.c]
float scalbnf(float x, int n);
long double scalbnl(long double x, int n);
```

```
short float scalbln(short float x, long int n);  // see [library.c]
```

```
float scalbln(float x, long int n);  // see [library.c]
double scalbln(double x, long int n);
long double scalbln(long double x, long int n);  // see [library.c]
float scalblnf(float x, long int n);
long double scalblnl(long double x, long int n);
```

```
short float cbrt(short float x);  // see [library.c]
```

```
float cbrt(float x);  // see [library.c]
double cbrt(double x);
long double cbrt(long double x);  // see [library.c]
float cbrtf(float x);
long double cbrtl(long double x);

// [c.math.abs], absolute values
int abs(int j);
long int abs(long int j);
long long int abs(long long int j);
```

```
short float abs(short float j);
```

```
float abs(float j);
double abs(double j);
long double abs(long double j);
```

```
short float fabs(short float x);  // see [library.c]
```

```
float fabs(float x);  // see [library.c]
double fabs(double x);
long double fabs(long double x);  // see [library.c]
float fabsf(float x);
long double fabsl(long double x);
```

```
short float hypot(short float x, short float y);  // see [library.c]
```

```
float hypot(float x, float y);  // see [library.c]
double hypot(double x, double y);
long double hypot(long double x, long double y);  // see [library.c]
float hypotf(float x, float y);
long double hypotl(long double x, long double y);

// [c.math.hypot3], three-dimensional hypotenuse
```

```
short float hypot(short float x, short float y, short float z);
```

```
float hypot(float x, float y, float z);
double hypot(double x, double y, double z);
long double hypot(long double x, long double y, long double z);
```

```
short float pow(short float x, short float y);  // see [library.c]
```

```
float pow(float x, float y);  // see [library.c]
double pow(double x, double y);
long double pow(long double x, long double y);  // see [library.c]
float powf(float x, float y);
long double powl(long double x, long double y);
```

```
short float sqrt(short float x);  // see [library.c]
```

```
float sqrt(float x);  // see [library.c]
double sqrt(double x);
long double sqrt(long double x);  // see [library.c]
float sqrtf(float x);
long double sqrtl(long double x);
```

```
short float erf(short float x);  // see [library.c]
```

```
float erf(float x);  // see [library.c]
double erf(double x);
long double erf(long double x);  // see [library.c]
float erff(float x);
long double erfl(long double x);
```

```
short float erfc(short float x);  // see [library.c]
```

```
float erfc(float x);  // see [library.c]
double erfc(double x);
long double erfc(long double x);  // see [library.c]
float erfcf(float x);
long double erfcl(long double x);
```

```
short float lgamma(short float x);  // see [library.c]
```

```
float lgamma(float x);  // see [library.c]
double lgamma(double x);
long double lgamma(long double x);  // see [library.c]
float lgammaf(float x);
long double lgammal(long double x);
```

```
short float tgamma(short float x);  // see [library.c]
```

```
float tgamma(float x);  // see [library.c]
double tgamma(double x);
long double tgamma(long double x);  // see [library.c]
float tgammaf(float x);
long double tgammal(long double x);
```

```
short float ceil(short float x);  // see [library.c]
```

```
float ceil(float x);  // see [library.c]
double ceil(double x);
long double ceil(long double x);  // see [library.c]
float ceilf(float x);
long double ceill(long double x);
```

```
short float floor(short float x);  // see [library.c]
```

```
float floor(float x);  // see [library.c]
double floor(double x);
long double floor(long double x);  // see [library.c]
float floorf(float x);
long double floorl(long double x);
```

```
short float nearbyint(short float x);  // see [library.c]
```

```
float nearbyint(float x);  // see [library.c]
double nearbyint(double x);
long double nearbyint(long double x);  // see [library.c]
float nearbyintf(float x);
long double nearbyintl(long double x);
```

```
short float rint(short float x);  // see [library.c]
```

```
float rint(float x);  // see [library.c]
double rint(double x);
long double rint(long double x);  // see [library.c]
float rintf(float x);
long double rintl(long double x);
```

```
long int lrint(short float x);  // see [library.c]
```

```
long int lrint(float x);  // see [library.c]
long int lrint(double x);
long int lrint(long double x);  // see [library.c]
long int lrintf(float x);
long int lrintl(long double x);
```

```
long long int llrint(short float x);  // see [library.c]
```

```
long long int llrint(float x);  // see [library.c]
long long int llrint(double x);
long long int llrint(long double x);  // see [library.c]
long long int llrintf(float x);
long long int llrintl(long double x);
```

```c
short float round(short float x);  // see [library.c]
```

```c
float round(float x);  // see [library.c]
double round(double x);
long double round(long double x);  // see [library.c]
float roundf(float x);
long double roundl(long double x);
```

```c
long int lround(short float x);  // see [library.c]
```

```c
long int lround(float x);  // see [library.c]
long int lround(double x);
long int lround(long double x);  // see [library.c]
long int lroundf(float x);
long int lroundl(long double x);
```

```c
long long int llround(short float x);  // see [library.c]
```

```c
long long int llround(float x);  // see [library.c]
long long int llround(double x);
long long int llround(long double x);  // see [library.c]
long long int llroundf(float x);
long long int llroundl(long double x);
```

```c
short float trunc(short float x);  // see [library.c]
```

```c
float trunc(float x);  // see [library.c]
double trunc(double x);
long double trunc(long double x);  // see [library.c]
float truncf(float x);
long double truncl(long double x);
```

```c
short float fmod(short float x, short float y);  // see [library.c]
```

```c
float fmod(float x, float y);  // see [library.c]
double fmod(double x, double y);
long double fmod(long double x, long double y);  // see [library.c]
float fmodf(float x, float y);
long double fmodl(long double x, long double y);
```

```c
short float remainder(short float x, short float y);  // see [library.c]
```

```c
float remainder(float x, float y);  // see [library.c]
double remainder(double x, double y);
long double remainder(long double x, long double y);  // see [library.c]
float remainderf(float x, float y);
long double remainderl(long double x, long double y);
```

```c
short float remquo(short float x, short float y, int* quo);  // see [library.c]
```

```c
float remquo(float x, float y, int* quo);  // see [library.c]
double remquo(double x, double y, int* quo);
long double remquo(long double x, long double y, int* quo);  // see [library.c]
float remquof(float x, float y, int* quo);
long double remquol(long double x, long double y, int* quo);
```

```
short float copysign(short float x, short float y);  // see [library.c]
```

```
float copysign(float x, float y);  // see [library.c]
double copysign(double x, double y);
long double copysign(long double x, long double y);  // see [library.c]
float copysignf(float x, float y);
long double copysignl(long double x, long double y);

double nan(const char* tagp);
float nanf(const char* tagp);
long double nanl(const char* tagp);
```

```
short float nextafter(short float x, short float y);  // see [library.c]
```

```
float nextafter(float x, float y);  // see [library.c]
double nextafter(double x, double y);
long double nextafter(long double x, long double y);  // see [library.c]
float nextafterf(float x, float y);
long double nextafterl(long double x, long double y);
```

```
short float nexttoward(short float x, long double y);  // see [library.c]
```

```
float nexttoward(float x, long double y);  // see [library.c]
double nexttoward(double x, long double y);
long double nexttoward(long double x, long double y);  // see [library.c]
float nexttowardf(float x, long double y);
long double nexttowardl(long double x, long double y);
```

```
short float fdim(short float x, short float y);  // see [library.c]
```

```
float fdim(float x, float y);  // see [library.c]
double fdim(double x, double y);
long double fdim(long double x, long double y);  // see [library.c]
float fdimf(float x, float y);
long double fdiml(long double x, long double y);
```

```
short float fmax(short float x, short float y);  // see [library.c]
```

```
float fmax(float x, float y);  // see [library.c]
double fmax(double x, double y);
long double fmax(long double x, long double y);  // see [library.c]
float fmaxf(float x, float y);
long double fmaxl(long double x, long double y);
```

```
short float fmin(short float x, short float y);  // see [library.c]
```

```
float fmin(float x, float y);  // see [library.c]
double fmin(double x, double y);
long double fmin(long double x, long double y);  // see [library.c]
float fminf(float x, float y);
long double fminl(long double x, long double y);
```

```
short float fma(short float x, short float y, short float z);  // see [library.c]
```

```
float fma(float x, float y, float z);  // see [library.c]
double fma(double x, double y, double z);
long double fma(long double x, long double y, long double z);  // see [library.c]
float fmaf(float x, float y, float z);
long double fmal(long double x, long double y, long double z);

// [c.math.fpclass], classification / comparison functions
```

```
int fpclassify(short float x);
```

```
int fpclassify(float x);
int fpclassify(double x);
int fpclassify(long double x);
```

```
bool isfinite(short float x);
```

```
bool isfinite(float x);
bool isfinite(double x);
bool isfinite(long double x);
```

```
bool isinf(short float x);
```

```
bool isinf(float x);
bool isinf(double x);
bool isinf(long double x);
```

```
bool isnan(short float x);
```

```
bool isnan(float x);
bool isnan(double x);
bool isnan(long double x);
```

```
bool isnormal(short float x);
```

```
bool isnormal(float x);
bool isnormal(double x);
bool isnormal(long double x);
```

```
bool signbit(short float x);
```

```
bool signbit(float x);
bool signbit(double x);
bool signbit(long double x);
```

```
bool isgreater(short float x, short float y);
```

```
bool isgreater(float x, float y);
bool isgreater(double x, double y);
bool isgreater(long double x, long double y);
```

```
bool isgreaterequal(short float x, short float y);
```

```
bool isgreaterequal(float x, float y);
bool isgreaterequal(double x, double y);
bool isgreaterequal(long double x, long double y);
```

```
  bool isless(short float x, short float y);
```

```
  bool isless(float x, float y);
  bool isless(double x, double y);
  bool isless(long double x, long double y);
```

```
  bool islessequal(short float x, short float y);
```

```
  bool islessequal(float x, float y);
  bool islessequal(double x, double y);
  bool islessequal(long double x, long double y);
```

```
  bool islessgreater(short float x, short float y);
```

```
  bool islessgreater(float x, float y);
  bool islessgreater(double x, double y);
  bool islessgreater(long double x, long double y);
```

```
  bool isunordered(short float x, short float y);
```

```
  bool isunordered(float x, float y);
  bool isunordered(double x, double y);
  bool isunordered(long double x, long double y);
```

[...]

Note: mathematical special functions for `short float` are not provided, out of concern about precision. They are still callable with a `short float` value thanks to a promotion to `float`.

Modify Header `<cmath>` synopsis **[cmath.syn]** paragraph 2:

2. For each set of overloaded functions within `<cmath>`, with the exception of `abs`, there shall be additional overloads sufficient to ensure:
   1. If any argument of arithmetic type corresponding to a `double` parameter has type `long double`, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `long double`.
   2. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has type `double` or an integer type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.
   3. Otherwise, ~~all arguments~~ if any argument of arithmetic type corresponding to a `double` ~~parameters have~~ parameter has type `float` ~~.~~ , then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `float`.
   4. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `short float`.

Modify Absolute values **[c.math.abs]** as follows:

1. [Note: The headers and declare the functions described in this subclause. — end note]

```
int abs(int j);
long int abs(long int j);
long long int abs(long long int j);
float abs(float j);
double abs(double j);
long double abs(long double j);
```

2. Effects: The abs functions have the semantics specified in the C standard library for the functions `abs`, `labs`, `llabs`, `fabsf`, `fabs`, and `fabsl`.
3. Remarks: If `abs()` is called with an argument of type `X` for which `is_unsigned_v<X>` is `true` and if `X` cannot be converted to `int` by integral promotion, the program is ill-formed. [Note: Arguments that can be promoted to `int` are permitted for compatibility with C. — end note]

```
short float abs(short float j);
```

4. Effects: as if by `static_cast<short float>(abs(static_cast<float>(j)))`.

Modify Three-dimensional hypotenuse [**c.math.hypot3**] as follows:

```
short float hypot(short float x, short float y, short float z);
float hypot(float x, float y, float z);
```
[...]

Modify Classification / comparison functions [**c.math.fpclass**] paragraph 1:

The classification / comparison functions behave the same as the C macros with the corresponding names defined in the C standard library. Each function is overloaded for the ~~three~~ four floating-point types.

Modify Class template `num_get` [**locale.num.get**], adding new overloads:

[...]

```
  iter_type get(iter_type in, iter_type end, ios_base&,
                ios_base::iostate& err, unsigned long long& v) const;

  iter_type get(iter_type in, iter_type end, ios_base&,
                ios_base::iostate& err, short float& v) const;

  iter_type get(iter_type in, iter_type end, ios_base&,
                ios_base::iostate& err, float& v) const;
```

[...]

```
  virtual iter_type do_get(iter_type, iter_type, ios_base&,
                           ios_base::iostate& err, unsigned long long& v) const;

  virtual iter_type do_get(iter_type, iter_type, ios_base&,
                           ios_base::iostate& err, short float& v) const;

  virtual iter_type do_get(iter_type, iter_type, ios_base&,
                           ios_base::iostate& err, float& v) const;
```

Modify `num_get` members [**facet.num.get.members**], mentioning the new overload:

[...]

```
iter_type get(iter_type in, iter_type end, ios_base& str,
              ios_base::iostate& err, unsigned long long& val) const;
```

```
iter_type get(iter_type in, iter_type end, ios_base& str,
              ios_base::iostate& err, short float& val) const;
```

```
iter_type get(iter_type in, iter_type end, ios_base& str,
              ios_base::iostate& err, float& val) const;
```

[...]

Modify `num_get` virtual functions [**facet.num.get.virtuals**], mentioning the new overload:

```
iter_type do_get(iter_type in, iter_type end, ios_base& str,
                 ios_base::iostate& err, unsigned long long& val) const;
```

```
iter_type do_get(iter_type in, iter_type end, ios_base& str,
                 ios_base::iostate& err, short float& val) const;
```

```
iter_type do_get(iter_type in, iter_type end, ios_base& str,
                 ios_base::iostate& err, float& val) const;
```

In `num_get` virtual functions [**facet.num.get.virtuals**] paragraph 3 stage 3, insert a new item before item 3:

[...]

- For an unsigned integer value, the function `strtoull`.
- For a `short float` value, the function `strtof`.
- For a `float` value, the function `strtof`.

[...]

Modify Class template `basic_istream` [**istream**], adding a new overload:

[...]

```
basic_istream<charT, traits>& operator>>(unsigned long long& n);
```

```
basic_istream<charT, traits>& operator>>(short float& f);
```

```
basic_istream<charT, traits>& operator>>(float& f);
```

[...]

Modify Arithmetic extractors [**istream.formatted.arithmetic**] to mention the new overload:

[...]

```
operator>>(unsigned long long& val);
operator>>(short float& val);
operator>>(float& val);
```

[...]

Modify Class template `basic_ostream` **[ostream]**, adding a new overload:

[...]

```
basic_ostream<charT, traits>& operator<<(unsigned long long n);
basic_ostream<charT, traits>& operator<<(short float f);
basic_ostream<charT, traits>& operator<<(float f);
```

[...]

Modify Arithmetic inserters **[ostream.inserters.arithmetic]** as follows:

```
operator<<(bool val);
operator<<(short val);
operator<<(unsigned short val);
operator<<(int val);
operator<<(unsigned int val);
operator<<(long val);
operator<<(unsigned long val);
operator<<(long long val);
operator<<(unsigned long long val);
operator<<(short float val);
operator<<(float val);
operator<<(double val);
operator<<(long double val);
operator<<(const void* val);
```

1. Effects: The classes `num_get<>` and `num_put<>` handle locale-dependent numeric formatting and parsing. These inserter functions use the imbued locale value to perform numeric formatting. When val is of type `bool`, `long`, `unsigned long`, `long long`, `unsigned long long`, `double`, `long double`, or `const void*`, the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<
  num_put<charT, ostreambuf_iterator<charT, traits>>
    >(getloc()).put(*this, *this, fill(), val).failed();
```

When val is of type `short` the formatting conversion occurs as if it performed the following code fragment:

```
ios_base::fmtflags baseflags = ios_base::flags() & ios_base::basefield;
bool failed = use_facet<
  num_put<charT, ostreambuf_iterator<charT, traits>>
    >(getloc()).put(*this, *this, fill(),
    baseflags == ios_base::oct || baseflags == ios_base::hex
      ? static_cast<long>(static_cast<unsigned short>(val))
      : static_cast<long>(val)).failed();
```

When val is of type `int` the formatting conversion occurs as if it performed the following code fragment:

```
ios_base::fmtflags baseflags = ios_base::flags() & ios_base::basefield;
bool failed = use_facet<
  num_put<charT, ostreambuf_iterator<charT, traits>>
    >(getloc()).put(*this, *this, fill(),
    baseflags == ios_base::oct || baseflags == ios_base::hex
      ? static_cast<long>(static_cast<unsigned int>(val))
      : static_cast<long>(val)).failed();
```

When val is of type `unsigned short` or `unsigned int` the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<
  num_put<charT, ostreambuf_iterator<charT, traits>>
    >(getloc()).put(*this, *this, fill(),
      static_cast<unsigned long>(val)).failed();
```

When val is of type `short float` or `float` the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<
  num_put<charT, ostreambuf_iterator<charT, traits>>
    >(getloc()).put(*this, *this, fill(),
      static_cast<double>(val)).failed();
```

Modify Specializations for floating-point types [**atomics.ref.float**] paragraph 1:

> There are specializations of the `atomic_ref` class template for the floating-point types `short float`, `float`, `double`, and `long double`. For each such type *floating-point*, the specialization `atomic_ref<floating-point>` provides additional atomic operations appropriate to floating-point types.

Modify Specializations for floating-point types [**atomics.types.float**] paragraph 1:

> There are specializations of the `atomic` class template for the floating-point types `short float`, `float`, `double`, and `long double`. For each such type *floating-point*, the specialization `atomic<floating-point>` provides additional atomic operations appropriate to floating-point types.

4.2. Wording for library aliases

4.2.1. Library wording

Modify Headers [**headers**], table 18, by adding the new header to the list of C++ headers:

> [...]
>
> `<contract>`
> `<cstdfloat>`
> `<deque>`
>
> [...]

Modify Freestanding implementation [**compliance**], table 21:

	Subclause	Header(s)
[...]	[...]	[...]
16.4	Integer types	`<cstdint>`
16.?	Floating-point types	`<cstdfloat>`
16.5	Start and termination	`<cstdlib>`
[...]	[...]	[...]

Modify General **[support.general]** table 34:

	Subclause	Header(s)
[...]	[...]	[...]
16.4	Integer types	`<cstdint>`
16.?	Floating-point types	`<cstdfloat>`
16.5	Start and termination	`<cstdlib>`
[...]	[...]	[...]

Insert a new subclause into Language support library **[language.support]** after Integer types **[cstdint]**:

16.? Floating-point types **[cstdfloat]** 16.?.1 Header `<cstdfloat>` synopsis **[cstdfloat.syn]**

```
namespace std {
  using float16_t = floating-point type; // optional
  using float32_t = floating-point type; // optional
  using float64_t = floating-point type; // optional
}
```

16.?.1.1 Exact-width floating-point types

1. The typedef name `std::floatX_t` designates a *floating-point type* with width X, no padding bits, and a representation conforming to that defined as `binaryX` format in ISO/IEC/IEEE 60559.
2. These types are optional. However, if an implementation provides floating-point types with widths of 16, 32, or 64 bits, no padding bits, and that have a representation conforming to that defined above, it shall define the corresponding typedef names.

§ References

§ Informative References

[ARM-HF]
Half-precision floating-point number support. URL: http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.dui0205j/CIHGAECI.html

[BFLOAT16]
bfloat16 floating-point format. URL: https://en.wikipedia.org/wiki/Bfloat16_floating-point_format

[CUDA7.5]
Mark Harris. New Features in CUDA 7.5. URL: https://devblogs.nvidia.com/new-features-cuda-7-5/

[DNG]
Adobe. Digital Negative (DNG) Specification. URL: http://wwwimages.adobe.com/www.adobe.com/content/dam/acom/en/products/photoshop/pdfs/dng_spec_1.4.0.0.pdf

[GCCFP16]
Half-Precision Floating Point. URL: https://gcc.gnu.org/onlinedocs/gcc/Half-Precision.html

[IEEE-754-2008]
IEEE Standard for Floating-Point Arithmetic. 29 August 2008. URL: http://ieeexplore.ieee.org/servlet/opac?punumber=4610933

[INTEL-HALF-PERF]

Patrick Konsor. Performance Benefits of Half Precision Floats. URL: https://software.intel.com/en-us/articles/performance-benefits-of-half-precision-floats

[LLVMIR]
LLVM Language Reference Manual. URL: http://releases.llvm.org/7.0.0/docs/LangRef.html#floating-point-types

[OpenEXR]
Florian Kainz; Rob Bogart; Piotr Stanczyk. Technical Introduction to OpenEXR. URL: http://openexr.com/documentation/TechnicalIntroduction.pdf

[OpenGL3.0]
Khronos Group. The OpenGL (R) Graphics System: A Specification (Version 3.0 - September 23, 2008). URL: https://www.khronos.org/registry/OpenGL/specs/gl/glspec30.pdf

[P0192R1]
Boris Fomitchev; et al. Adding Fundamental Type for Short Float. URL: https://wg21.link/P0192R1

§ Issues Index

ISSUE 1 it is possible that instead of adding a new overload, the overloads could be folded into a single specified function, the way that integer overloads of all of these are specified. ↵

ISSUE 2 the definitions of the `sto*` family depend on C functions in the `strto*` family. Should an overload for `short float` be added? ↵